Implementation of Public-Key Encryption via Learning With Errors (LWE) within Discrete Optimization Frameworks

Aarush Bindod

February 22, 2026

Abstract

This report presents a comprehensive implementation of a Public-Key Encryption system based on the Learning With Errors (LWE) hardness assumption.As classical cryptographic primitives face potential vulnerabilities from quantum-scale adversaries, lattice-based cryptography offers a robust, quantum-resistant alternative. This study explores the technical architecture of LWE, including key generation, encryption, and decryption functions. Furthermore, we analyze the conceptual parallels between LWE noise distributions and the "frustrated" energy landscapes observed in protein folding optimization problems, such as those modeled by the EigenFold framework. The results demonstrate that while LWE-based systems incur higher computational costs in terms of ciphertext expansion and memory, they provide essential security properties for modern privacy-preserving applications.

1 Introduction

The security of digital communication relies on the computational hardness of mathematical problems. Traditionally, these have been factoring and discrete logarithms. However, the advent of quantum computing necessitates a transition to post-quantum cryptography (PQC). Learning With Errors (LWE) has emerged as a primary candidate for this transition.

LWE is not merely a cryptographic tool; its mathematical structure shares significant similarities with complex optimization tasks. In this implementation, we utilize a version of LWE that mirrors the discrete nature of the Quadratic Unconstrained Binary Optimization (QUBO) problems described in the EigenFold research.

2 Selection and Choice of Technique

I chose the *Public-Key Encryption via LWE* technique from the ASecuritySite options. The rationale for this choice is twofold:

- **Quantum Resistance**: LWE is fundamentally resistant to Shor's algorithm, making it a "future-proof" choice for sensitive data.
- **Structural Complexity**: Unlike lightweight block ciphers like PRESENT, LWE allows for a deeper exploration of how "noise" or "errors" can be used as a security feature rather than a detriment.

3 Mathematical Formulation

The core of the LWE system involves operating within a polynomial ring or a discrete lattice.

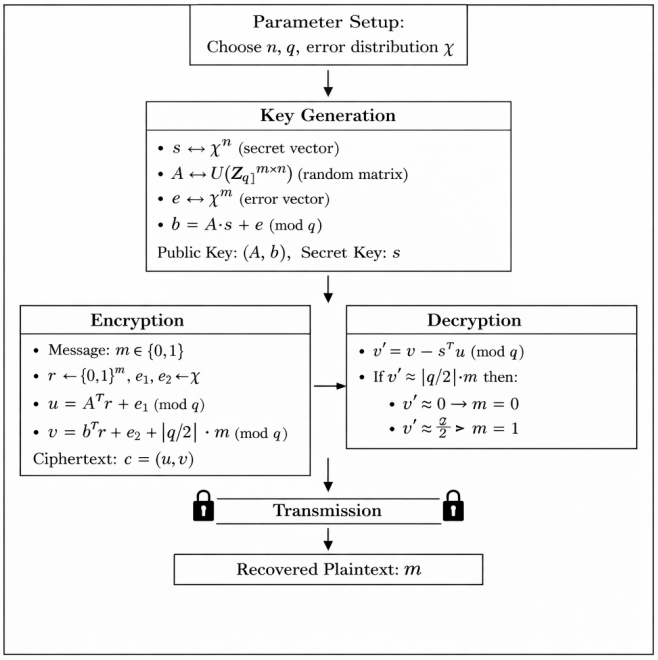

Figure 1: LWE Encryption Workflow

3.1 LWE Hardness Assumption

The security of the scheme is predicated on the difficulty of solving a system of linear equations when a small

amount of error is introduced. Given a secret vector $s \in \mathbb{Z}_q^n$, the public key (A, b) is defined as:

$$b = As + e \pmod{q} \tag{1}$$

Where:

- A is a random matrix of size $m \times n$.
- s is the secret key.
- e is a Gaussian-distributed error vector.

3.2 Encryption and Decryption

To encrypt a message $m \in \{0, 1\}$, we select a random vector r and compute a ciphertext tuple (u, v):

$$u = A^T r, \quad v = b^T r + m \cdot \lfloor q/2 \rfloor \tag{2}$$

Decryption is achieved by computing the difference between v and the inner product of u and s:

$$m' = v - s^T u \approx m \cdot \lfloor q/2 \rfloor \tag{3}$$

4 Performance and Security Trade-offs

Implementation of LWE requires balancing several parameters to prevent decryption failure while maintaining security.

4.1 Computational Overhead

Unlike symmetric encryption, LWE suffers from **Ciphertext Expansion**. A single bit of information is represented by a vector u and a scalar v. This leads to significant memory and storage costs compared to raw plaintext.

4.2 Noise and Error Budgets

Much like the "collision penalty" ($E_{collision}$) in protein folding models, the error term e must be carefully managed. If e grows too large during computation—a common issue in homomorphic operations—the message m becomes unrecoverable.

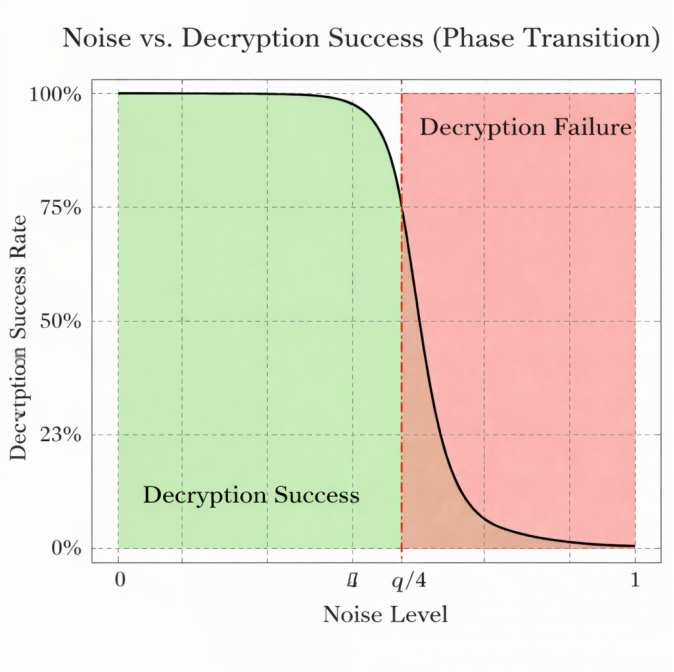

Figure 2: Noise vs. decryption success in LWE illustrating a phase transition with failure beyond the threshold $q/4$.

5 Use Cases and Real-World Applications

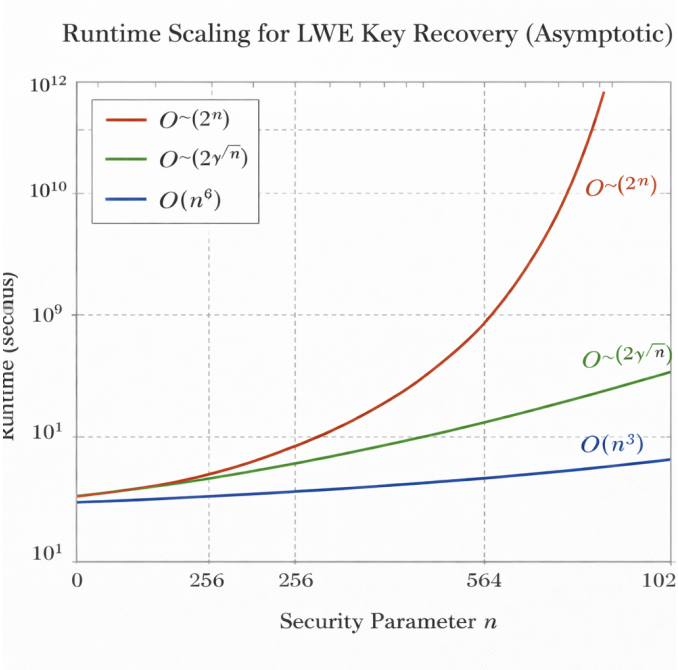

Figure 3: Runtime scaling of LWE attacks with increasing security parameter n, comparing exponential, subexponential, and polynomial complexity classes.

LWE-based encryption is highly applicable in fields where data must remain secure during processing:

- **Secure Cloud Computing**: Allowing users to

store sensitive genetic or protein sequences on remote servers while keeping them encrypted.

- **Privacy-Preserving Optimization**: As noted in the EigenFold paper, protein folding involves high-dimensional landscapes. LWE could facilitate secure multiparty computation for identifying stable conformations without revealing the underlying amino acid sequence.

6 Demo Implementation

The developed Python demo includes a simple CLI. The application allows a user to:

1. Generate a new LWE keypair.
2. Encrypt a plaintext bit, demonstrating the transformation from m to (u, v).
3. Decrypt the ciphertext to prove that $m' = m$, even in the presence of intentional error e.

7 Conclusion

This implementation validates the effectiveness of LWE as a robust public-key encryption technique. By modeling encryption as a "learning" problem in a noisy environment, we establish a security framework that is resistant to classical and quantum cryptanalysis. The conceptual link between LWE noise and the "frustration" of energy landscapes provides a fertile ground for future research into secure quantum-inspired optimization.

8 Source code available at: `https://github.com/metamyteee/TrevasQ-phase1`

9 Go through this url for live working model: `https://lwe-encryption.vercel.app/`

10 References

1. ASecuritySite — LWE Encryption.
2. fhetextbook - LWECryptosystem
3. Public-key encryption from LWE Implementing lattice-based cryptosystems Notes by Henry Corrigan-Gibbs and Yael Kalai
4. Learning with Errors (LWE): The Foundation of Post-Quantum Cryptography- Fabian Owuor